

Motivation

Freshman scheduling at large universities requires assigning thousands of students to hundreds of sections under **program**, **capacity**, **math placement**, and **time constraints**. Manual approaches don't scale and lack flexibility for custom administrative policies. Existing constraint programming systems require domain experts to formulate constraints manually.

Dataset: Synthetic dataset based on **RIT academic structure**, modeling 67 selected freshman-entry programs; sections, timings, and enrollments were synthesized.

Contribution: An automated system that combines **LLM-powered natural language constraint generation** (Claude Sonnet API) with CP-SAT optimization (Google OR-Tools), enabling non-technical administrators to express scheduling policies in plain English.

System Architecture

• **Data Input:** 3 CSV files (sections, requirements, cohorts).

• **Data Validation:** feasibility analysis, N-way conflict detection, effective capacity checks.

• **LLM Interface:** natural language → CP-SAT code via Claude Sonnet API.

• **CP-SAT Engine:** binary assignment variables with hard and soft constraints.

• **Power BI Dashboard:** interactive visualization of enrollment and utilization.

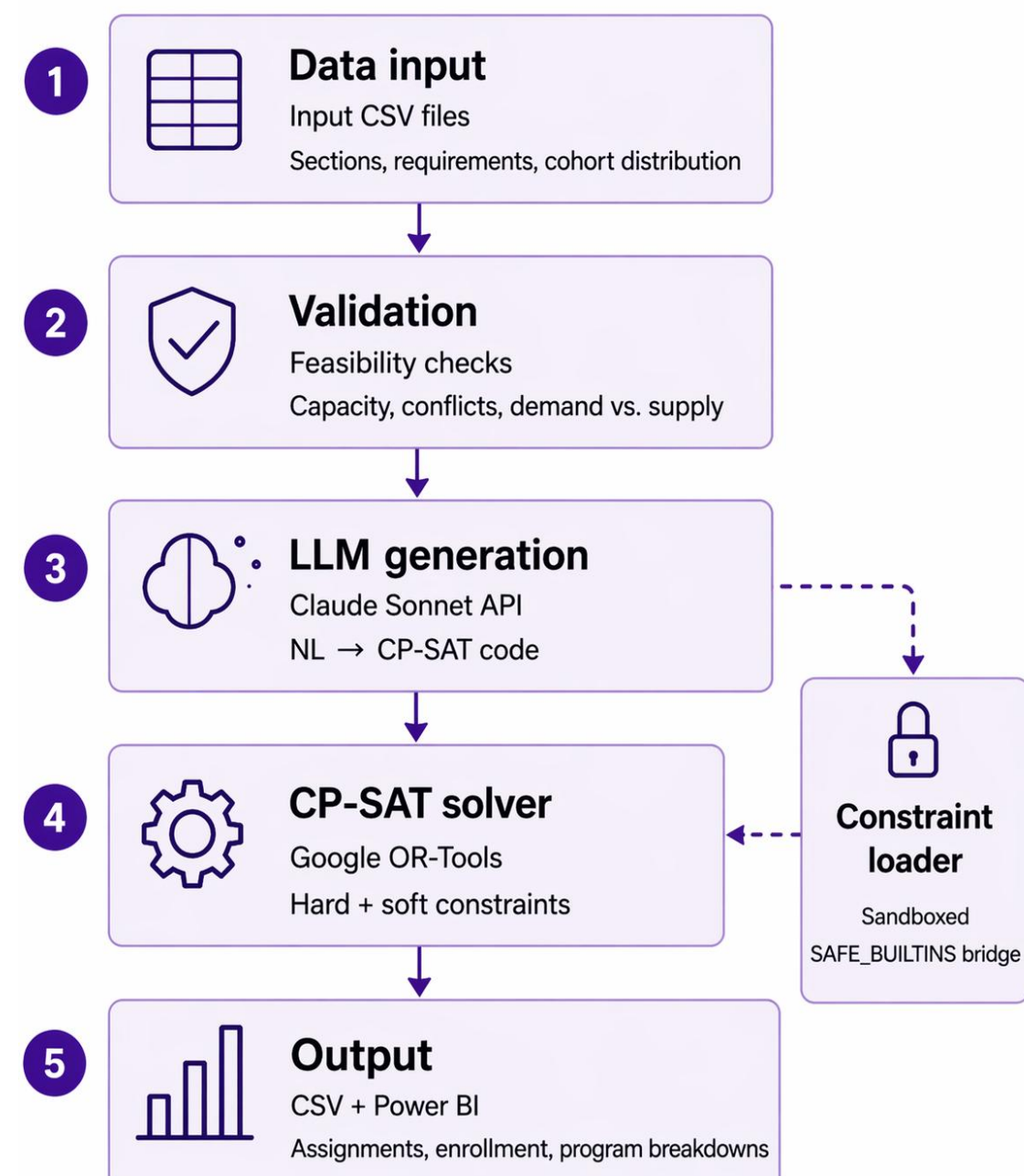


Fig 1. Automated Scheduling Framework

Decision Variable

$x_{s,c,k} = 1$ if student s is assigned to section k of course c

Hard 1: One section

Each student takes exactly one section per required course

Hard 2: No conflicts

A student cannot be assigned to conflicting sections

Hard 3: Capacity

Enrollment in any section cannot exceed its capacity

Objective: Balance

Minimize imbalance in section enrollments (fair distribution)

s : student, c : course, k : section, R_s : courses required by student s , $cap_{c,k}$: capacity of section k in course c , K_c : total number of sections for course c , w : weighting factor

$$x_{s,c,k} \in \{0, 1\} \quad (\text{Decision variable: 1 if student } s \text{ is assigned to section } k \text{ of course } c)$$

$$\sum_{k=0}^{K_c-1} x_{s,c,k} = 1 \quad \forall s, \forall c \in R_s \quad (\text{One section per required course})$$

$$x_{s,c_1,k_1} + x_{s,c_2,k_2} \leq 1 \quad \forall s, \forall (c_1, k_1), (c_2, k_2) \text{ conflict} \quad (\text{No time conflicts})$$

$$\sum_{s \in R_s} x_{s,c,k} \leq cap_{c,k} \quad \forall c, \forall k \quad (\text{Respect section capacity})$$

$$\text{Minimize } \sum_{c: |K_c| \geq 2} w \cdot \left(\max_c - \min_c \right) \quad (\text{Balance section enrollments})$$

Fig 2. Optimization Model Formulation

LLM Constraint Generation

Administrators describe constraints in **natural language**; Claude Sonnet translates to executable CP-SAT code.

Supports: percentage caps, even distribution, multi-program groups, exclusion rules, minimum enrollment.

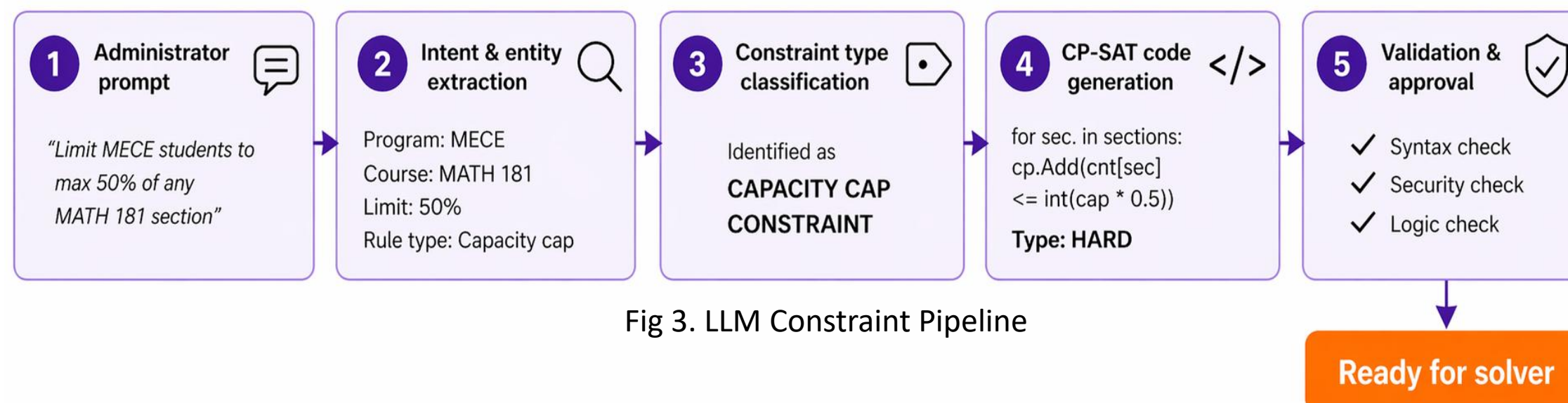


Fig 3. LLM Constraint Pipeline

Constraint Examples

INPUT — Natural Language

“Distribute GCCIS students evenly across all sections”

OUTPUT — CP-SAT Code

```
1 # Soft constraint
2 for sec in sections:
3     diff = count - avg
4     objective_terms.append(diff)
```

Type: **SOFT** optimized, not mandatory

INPUT — Natural Language

“Limit MECE students to max 50% of any MATH 181 section”

OUTPUT — CP-SAT Code

```
1 # Hard constraint
2 for sec in sections:
3     cap = int(cap[s] * 0.5)
4     model.Add(mece <= cap)
```

Type: **HARD** enforced absolutely

Results & Verification

Key Results					
Metric	Value	Metric	Value	Metric	Value
Students Assigned	4,155	Sections Used	47	Time Conflicts	0
Syntax Validity	100%	Capacity Violations	0	Decision Variables	~317K
Solver Configuration	300s, 8 workers	Constraint Types	Hard + Soft	Infeasible Detection	Yes

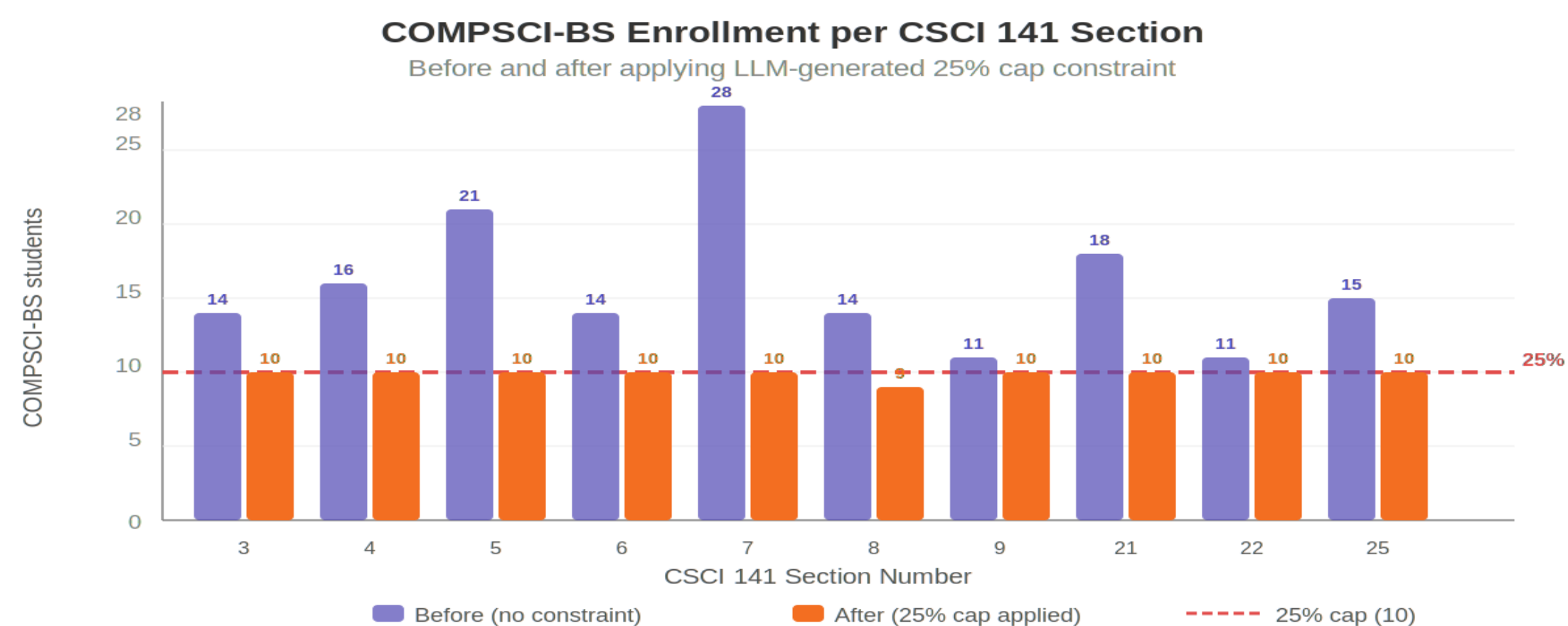
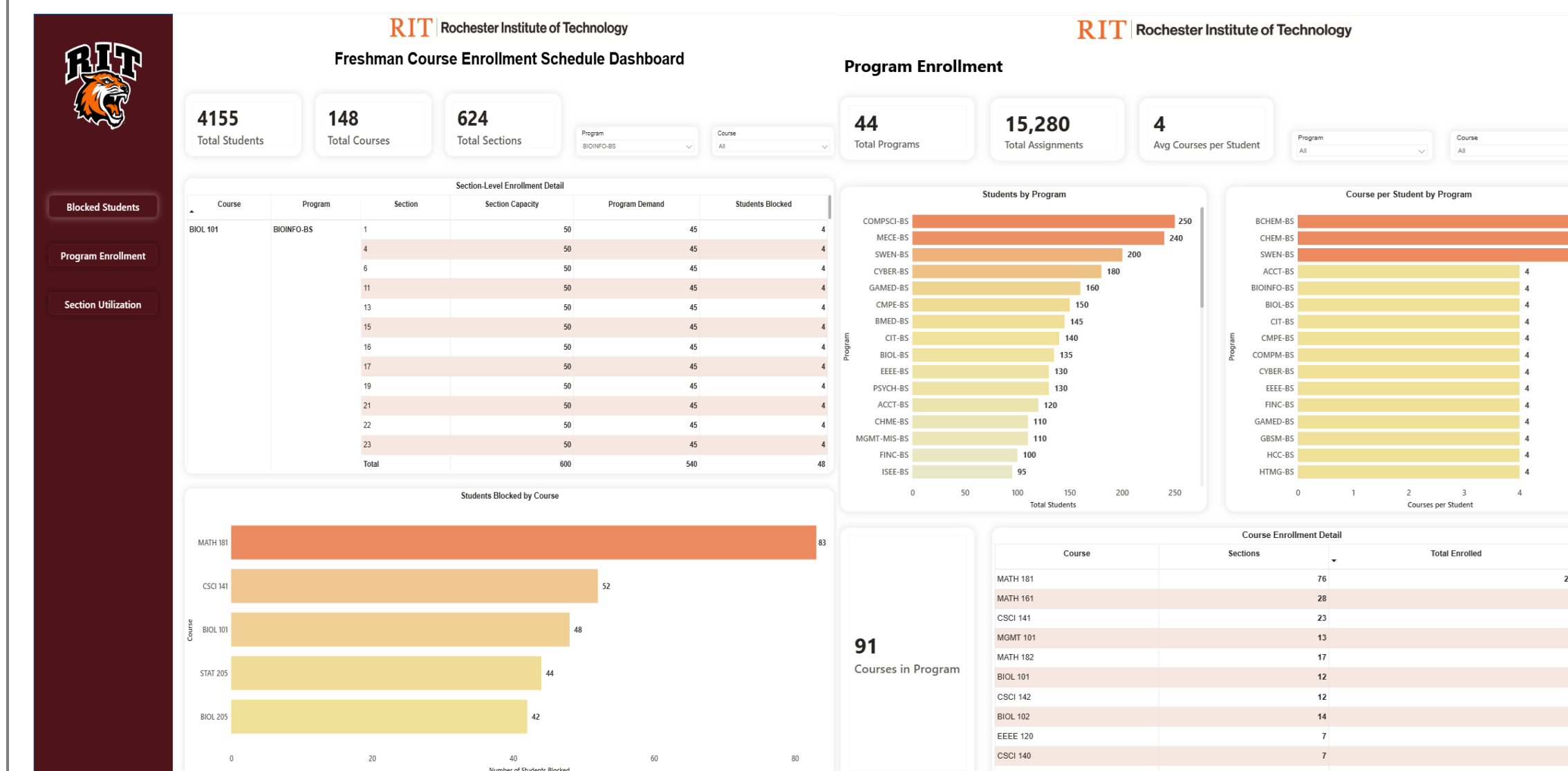


Fig 4. Constraint Impact on Section Enrollment

100% feasible schedules with no conflicts, balanced section loads, and successful execution of LLM-generated policies.

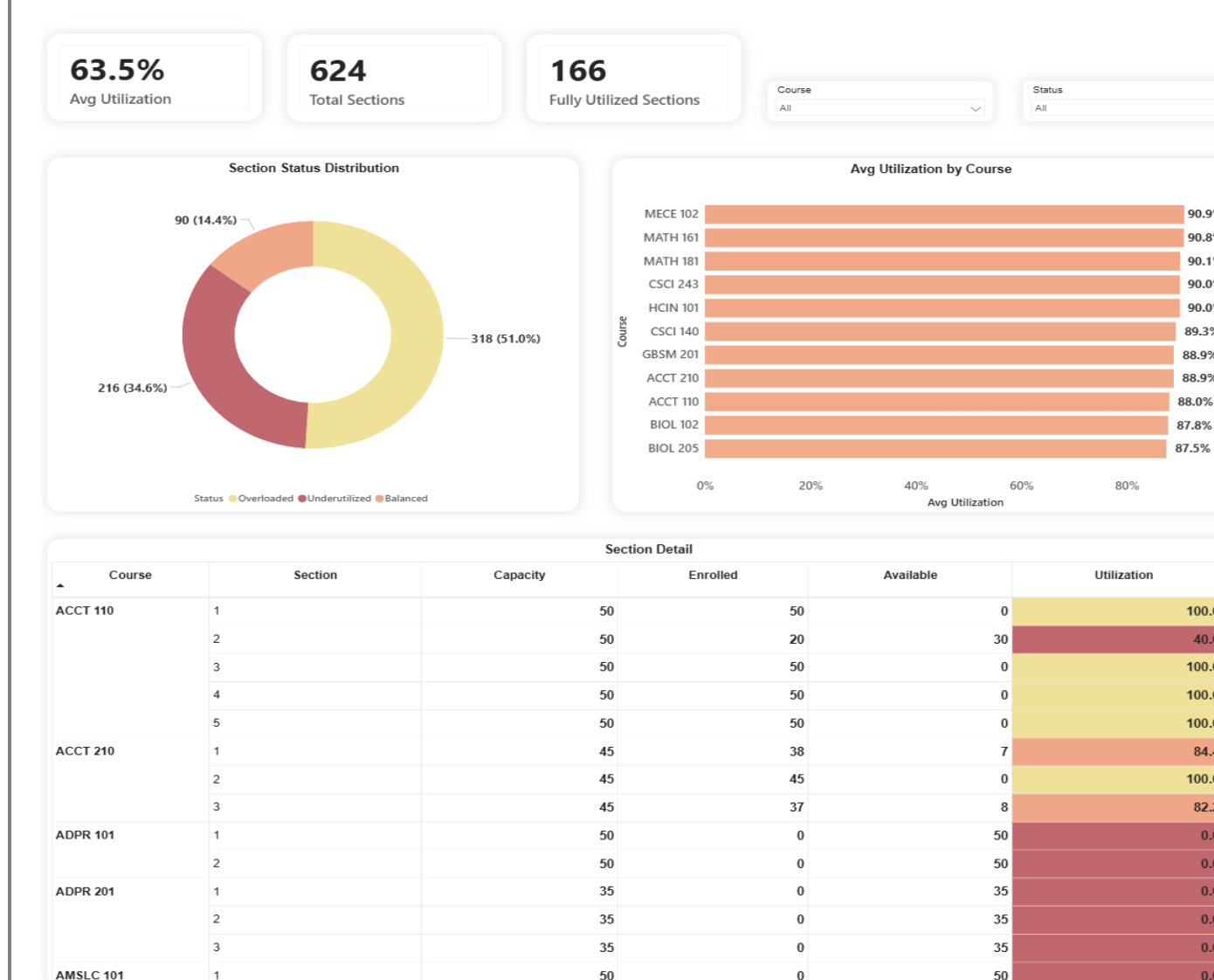
Power BI Dashboard



Dashboard View 1. Enrollment Overview

Dashboard View 2. Program Distribution

Section Utilization



Dashboard View 3: Section Utilization

Conclusions & Future Work

Summary: This project demonstrates that LLM-generated constraints can be safely integrated into a CP-SAT scheduling pipeline through sandboxed execution and two-layer security validation. Evaluated on synthetic data modeled on RIT's program structure, the system produced 100% feasible schedules with zero-time conflicts, correctly detected infeasible constraint combinations, and achieved 100% syntax validity on LLM-generated code.

Future Work: Future development includes deployment on **real RIT enrollment data**, development of a web-based administrator interface beyond the current CLI, modeling of student preferences such as instructor and time-slot choices, and joint optimization of student-section and section-room assignments.

References

- Google, “OR-Tools: Google Optimization Tools,” 2024. Available: <https://developers.google.com/optimization>
- S. Ceschia, L. Di Gaspero, and A. Schaerf, “Educational timetabling: Problems, benchmarks, and state-of-the-art results,” European Journal of Operational Research, vol. 308, no. 1, pp. 1–18, 2023.